Roteiro de Prática Guiada: Docker Compose & GitHub Actions para o PI

Integração e Entrega Contínua — Fatec Jahu
Prof. Dr. Leandro Sebastian Pereira da Silva

Apresentação Pedagógica

Na aula anterior, aprendemos a criar e rodar individualmente containers de bancos de dados utilizando comandos extensos do terminal. No mercado e na construção do seu Projeto Integrador (PI), gerir múltiplos containers (Aplicação, Banco de Dados, Autenticação) de forma manual torna-se inviável e propenso a erros.

Este roteiro introduz duas ferramentas essenciais da cultura DevOps:

- **Docker Compose:** Gerencia a infraestrutura como código (IaC), permitindo inicializar toda a arquitetura do seu grupo com um único comando.
- **GitHub Actions (CI/CD):** Automatiza os testes e a integração do código na nuvem a cada atualização enviado para o repositório.

Passo 1: Preparação do Ambiente de Trabalho

Antes de escrever as configurações, precisamos garantir o ambiente correto no editor.

- **1. Inicialize o Docker Desktop:** Certifique-se de que a baleia na barra de tarefas está verde sólida (Engine Running).
- **2. Abra o VS Code:** Vá em File -> Open Folder... e crie uma nova pasta chamada pi-automacao.
- **3. Abra o Terminal Integrado:** Pressione Ctrl + ' (ou Terminal -> New Terminal). Garanta que está utilizando o perfil Git Bash ou Bash nativo para maior compatibilidade com comandos Linux.

Passo 2: O Dockerfile da Aplicação

O Dockerfile é o arquivo que define a infraestrutura interna da aplicação do seu grupo. Vamos criar uma imagem customizada baseada em Node.js para simular o servidor do PI.

Na barra lateral do VS Code, crie um arquivo chamado Dockerfile (sem qualquer extensão) e adicione o seguinte conteúdo:

```
FROM node:20-alpine
WORKDIR /app
```

```
EXPOSE 3000
CMD ["echo", "Aplicacao do PI conectada com sucesso!"]
```

O que está acontecendo por trás do código?

- **FROM node:20-alpine:** Diz ao Docker para baixar uma distribuição Linux ultra-enxuta que já vem com o runtime do Node.js pronto, economizando espaço em disco.
- **WORKDIR /app:** Determina que todas as ações seguintes ocorrerão dentro do diretório /app do container.
- **EXPOSE 3000:** Informa que o container abrirá internamente a porta de comunicação 3000.
- **CMD [...]:** Define o comando que será disparado assim que o container iniciar. Aqui estamos apenas printando uma validação na tela.

Passo 3: A Escolha do Banco de Dados do PI

Escolha apenas uma das três opções abaixo, de acordo com o banco de dados que o seu grupo adotou para o PI.

OPÇÃO 1 — Ecossistema PostgreSQL

1. Crie o arquivo init.sql na raiz do projeto para criar a tabela inicial estruturada:

```
CREATE TABLE usuarios (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(100),
  email VARCHAR(100) UNIQUE
);
INSERT INTO usuarios (nome, email) VALUES ('Admin PI',
'admin@fatec.sp.gov.br');
```

2. Crie o arquivo docker-compose.yml na raiz do projeto e insira a orquestração abaixo:

```
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      - DB_HOST=db
      - DB_USER=admin
      - DB_PASSWORD=senha123
      - DB_NAME=meubanco
    depends_on:
      - db

  db:
```

```
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: senha123
      POSTGRES_DB: meubanco
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  pg-data:
```

OPÇÃO 2 — Ecossistema MySQL

1. Crie o arquivo init.sql na raiz do projeto para criar a tabela inicial estruturada:

```
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100),
  email VARCHAR(100) UNIQUE
);
INSERT INTO usuarios (nome, email) VALUES ('Admin PI',
'admin@fatec.sp.gov.br');
```

2. Crie o arquivo docker-compose.yml na raiz do projeto e insira a orquestração abaixo:

```
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASSWORD=senha123
      - DB_NAME=meubanco
    depends_on:
      - db

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: senha123
      MYSQL_DATABASE: meubanco
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
```

```
volumes:
  mysql-data:
```

OPÇÃO 3 — Ecossistema MongoDB

1. Crie o arquivo init.js na raiz do projeto para criar o documento inicial:

```
db = db.getSiblingDB('meubanco');
db.usuarios.insertOne({
  nome: "Admin PI",
  email: "admin@fatec.sp.gov.br"
});
```

2. Crie o arquivo docker-compose.yml na raiz do projeto e insira a orquestração abaixo:

```
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      -
MONGO_URI=mongodb://admin:senha123@db:27017/meubanco?authSource=admin
    depends_on:
      - db

  db:
    image: mongo:6
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: senha123
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
      - ./init.js:/docker-entrypoint-initdb.d/init.js

volumes:
  mongo-data:
```

Aprendizado do arquivo Docker Compose

- **services:** Contém os containers que formam o ecossistema do seu PI. Cada serviço vira um container isolado em memória.
- **build: . :** Avisa que o Docker Compose deve buildar a aplicação local usando o Dockerfile contido na pasta atual.
- **ports:** Mapeia as portas. O formato indica MáquinaHospedeira:Container. Acessar localhost:8080 redirecionará o tráfego para a porta 3000 interna da aplicação.

- **depends_on:** Garante a ordem de subida. A aplicação só começa a ser inicializada depois que o container de banco de dados estiver operacional.
- **volumes:** Evita perda de dados. Ao mapear um volume declarado no final do arquivo, todas as inserções de dados no banco persistirão no seu HD real mesmo que o container seja destruído.

Passo 4: Execução e Validação Local

Hora de validar a infraestrutura local sem digitar linhas extensas de comandos.

1. No terminal do VS Code, execute o comando de orquestração:

```
docker compose up --build
```

Explicação: O Docker Compose lerá o seu arquivo YAML, baixará a imagem oficial do banco escolhido, construirá a imagem da sua aplicação do PI, criará uma rede virtual isolada para que ambos se comuniquem e montará o volume persistente de forma 100% automatizada.

2. Observe os Logs do Terminal:

Você verá mensagens do banco subindo. Adicionalmente, notará que o container da aplicação executará e sairá com o código 'Exited (0)'. Fique tranquilo: isso não é um erro! Como configuramos apenas um comando simples 'echo' no Dockerfile, ele cumpre o seu papel de escrita e encerra por segurança. Em aplicações reais, servidores HTTP (Express, Spring, Flask) prendem o terminal mantendo o status ativo.

3. Validando os status em outro terminal:

```
docker compose ps -a
```

Explicação: Este comando lista de forma focada apenas os containers que pertencem a este ecossistema do projeto, exibindo quem está ativo (Up) e quem encerrou com sucesso (Exited).

4. Limpando a bancada de trabalho:

Pressione Ctrl + C para parar o monitoramento e limpe os containers digitando:

```
docker compose down
```

Passo 5: Integração Contínua com GitHub Actions

Agora que o projeto funciona localmente, vamos garantir que qualquer modificação enviada pela equipe do PI seja validada automaticamente na nuvem do GitHub, garantindo que ninguém quebrou a infraestrutura básica do projeto.

1. Criação das pastas de automação:

Na raiz do seu projeto, crie uma pasta chamada .github e, dentro dela, crie uma subpasta chamada workflows (Atenção redobrada à grafia: o ponto inicial e o 's' final são obrigatórios).

2. Criação do arquivo de esteira de automação:

Dentro da pasta workflows, crie o arquivo ci.yml e insira o código:

```
name: Pipeline de CI do PI

on:
  push:
    branches: [ main, master ]

jobs:
  build-test:
    runs-on: ubuntu-latest

    steps:
    - name: Baixar codigo fonte
      uses: actions/checkout@v3

    - name: Testar Build da Imagem Docker
      run: docker build -t imagem-teste-pi .
```

O que o GitHub Actions fará na nuvem?

- **Gatilho:** O gatilho 'on: push' interceptará qualquer envio de código direcionado para as branches principais.
- **Ambiente de Execução:** O GitHub disponibilizará de forma gratuita uma máquina virtual Linux zerada (ubuntu-latest) nos servidores deles para executar as tarefas do grupo.
- **Passo 1 (Checkout):** O comando checkout baixa todo o código do repositório dos alunos para dentro desta máquina virtual temporária.
- **Passo 2 (Docker Build):** A máquina do GitHub executará o comando de build no Dockerfile criado no Passo 2. Se houver qualquer erro de digitação ou dependência ausente, o pipeline falhará imediatamente, alertando o grupo.

Passo 6: Disparando o Pipeline na Nuvem

Chegou o momento de ver a Integração Contínua acontecer.

1. Inicialize e faça o commit dos seus arquivos locais no terminal:

```
git init
git add .
git commit -m "Feat: Infraestrutura com Compose e esteira de CI
configurada"
```

```
# 1. git init: Inicializa o repositório Git na sua pasta local
# 2. git add .: Adiciona todos os arquivos criados (Dockerfile, docker-compose.yml, .github/workflows/ci.yml) para o monitoramento do Git
# 3. git commit -m "Feat: Infraestrutura com Compose e esteira de CI configurada": Salva o estado atual do projeto com uma mensagem explicativa
```

2. Vincule ao seu GitHub:

- **A)** Vá no seu navegador, acesse o GitHub e crie um novo repositório público chamado pi-automacao.
- **B)** Copie os comandos de vinculação exibidos na página oficial do repositório e execute-os no seu terminal. O formato padrão segue o modelo abaixo:

```
git remote add origin https://github.com/SEU_USUARIO/pi-automacao.git
git branch -M main
git push -u origin main
```

3. Acompanhe a Automação:

- **A)** Abra o repositório no navegador.
- **B)** Clique na aba Actions.
- **C)** Clique no fluxo chamado 'Pipeline de CI do PI' que está rodando. Você verá em tempo real a máquina virtual do GitHub lendo o Dockerfile do grupo e executando o build.

Se tudo foi configurado corretamente, o fluxo terminará com um Check Verde (✓), validando a entrega do grupo!